

COMP4121 Project 6 Report: Polymorphic Types

MAKSIMOVICH, Roman

rmaksimovich@connect.ust.hk

WU, Yiu Tsz

ytwuac@connect.ust.hk

1. Introduction

In its existing implementation, the Amy language lacks two prominent features of functional languages: lambda abstractions and parametric polymorphism. We have chosen to implement the latter, i.e. to support

- polymorphic types of the form $T[A_1, \dots A_n]$, where the type variables A_k may be instantiated to concrete types and constructors extending T may depend on A_k in the types of their arguments;
- polymorphic functions of the form $f[A_1, \dots A_n] (\dots)$, whose argument types and return type may depend on $A_1, \dots A_n$.

In addition, we have undertaken and completed the extra challenge of rewriting the entire compiler frontend from scratch in Rust, including

1. A naïve implementation of the lexer, without reliance on regular expressions;
2. A recursive-descent implementation of the parser;
3. A custom implementation of the name analyzer;
4. A custom implementation of the type-checker, with support for polymorphism;
5. A custom implementation of the interpreter.

Due to time constraints, we have chosen to skip the code generation stage and only implement the interpreter.

Our projects provides, apart from the extra language features, a state-of-the-art error report system, emitting messages that look like this:

Input:

```
1 object Test
2   abstract class Option[A]
3
4   case class None() extends Option
5   case class Some(x: A) extends W
6 end Test
```

Output:

```
Error during the resolving stage.
-> ./Test.amy:5:14
4       case class None() extends Option
5       case class !! Some(x: A) extends W !!
6       end Test
```

```
Could not find a type named `W` in the
`Test` module.
```

2. Examples

Consider the following module which defines a `List[A]` type:

```
object TL
  abstract class List[A]
  case class Nil() extends List
  case class Cons(h: A, t: List[A]) extends List

  def switchType[A,B] (xs: List[A]): List[B] :=
    xs match {
      case Nil() => Nil()
      case Cons(_, _) => error("no way to produce B from A.")
    }
  end switchType

  def length[C] (lst: List[C]): Int(32) :=
    lst match {
      case Nil() => 0
      case Cons(h, t) => 1 + length(t)
    }
  end length

  val xs: List[Int(32)] = Cons(3, Cons(7, Nil()));
  val ys: List[Boolean] = switchType(xs);
  ys
end TL
```

This program type-checks and the type variable A in List[A] is specified to different types in different scenarios. The following program, on the other hand, will not type-check:

```
object Test
  abstract class Triple[A, B, C]
  case class MkTriple(x: A, y: B, z: C) extends Triple

  def getFirst[X, Y, Z] (t: Triple[X, Y, Z]): X :=
    t match {
      case MkTriple(x, y, z) => y
    }
  end getFirst
end Test
```
